Hostel Complaint & Attender Duty Tracking System

Phase 1 — Design Document

Ajoy Mathew • CS23B101 • CS5013 Programming with AI

Stakeholder: Hostel Manager, Cauvery Hostel, IIT Madras • 11 September 2026

Scope of this document. A design summary. The full design — complete DDL, the 17-row transition table, API contracts, ADRs and the sprint plan — lives in the project repository under `docs/design/`. Section numbers here map onto it.

1 Problem & Scope

The Hostel Manager at Cauvery Hostel processes daily maintenance complaints entirely by hand. Residents submit a Google Form; the manager copies each response into a spreadsheet, sorts it into a domain (Electrical, Civil, RO Plant), checks which of four attenders covers that floor and is on duty, assigns the work verbally, chases inspection dockets, and at month end tallies points for bonus calculation. This costs **30–45 minutes every day** and is error-prone at every step.

Points accrue at **+2** per resolved complaint, custom values for special duties, and **−1** per logged infraction — currently reconciled in a spreadsheet that is easily corrupted when edited from mobile devices.

1.1 In scope

S1. **Automated ingestion** from the Google Form response sheet, with a CSV upload fallback, and no duplicate complaints ever created.
S2. **Categorization** into Electrical / Civil / RO Plant / Network, with a confidence gate that routes uncertain text to a manager triage queue rather than guessing.
S3. **Heuristic dispatch** to an attender, scoring floor coverage, live availability and current workload.
S4. **Field verification**: the attender records the IITM EU CMS docket number, which is a precondition for earning points.
S5. **Resolution workflow** with a formal state machine, and **points** on an append-only ledger (+2 resolutions, custom special duties, −1 penalties).
S6. **Push notifications** to attender phones when work is assigned or escalated.
S7. **Manager dashboard**: live counts, triage, leaderboard, MTTR, bottlenecks, anomaly alerts, and one-click `.xlsx` export.
S8. **Audit**: every state change and point movement reconstructible.

1.2 Out of scope

- **Resident accounts.** Residents keep using the existing Google Form; they never log in.
- **Replacing or integrating with the IITM EU CMS app.** No API is available to us. We store its docket number for traceability but can never verify it upstream.
- **A native mobile app.** Attenders use an installable web app (PWA); push arrives through it, not a Play Store binary.
- **Multi-hostel tenancy.** Single-hostel deployment; the schema models no hostel dimension.
- **Payroll.** We compute points; money is handled outside the system.

1.3 Actors

Actor	Count	Device	Responsibilities
Hostel Manager	1	Desktop	Oversight, triage, reassignment, penalties, special duties, exports, account provisioning
Attender	∼4	Android phone	Receive push alerts, work assigned queue, record CMS docket, resolve, claim duties, view own standing
Resident	hundreds	any	Submits the Google Form. *Never authenticates.*
Scheduler	—	—	Polls the sheet, ages priorities, runs nightly telemetry scans

1.4 Scale envelope

≤50 complaints/day, ≤10 concurrent users, ∼50k rows/year. Every sizing decision in this document follows from these numbers — and several designs were rejected precisely because they solved a scale problem this system will never have.

2 Authentication & Access Control

Google sign-in for attenders, a local password for the manager, invite-only provisioning for both. There is no self-registration: every principal is a pre-provisioned `app_user` row created by the manager.

	Manager	Attender
Mechanism	Username + password (form login)	Google OIDC (`openid, email, profile`)
Credential stored	BCrypt hash, cost 12	**None**
Admission decided by	Password check	Verified email present and active in `app_user`
Re-authentication	Type the password	One tap — account already signed in on Android

2.1 Why the split

Attenders get the mechanism with **no credential to lose, forget or leak**, and re-entry costs a single tap on a phone where the Google account is already active — better in the hand than a PIN, not merely cheaper to build. The manager gets the mechanism with **no external dependency**, because they are the only person who can reach the standby during a cloud outage, theirs is the account that must work in a demo whatever the network does, and they are the support desk that fixes everyone else's access.

The allowlist is the access-control boundary. Google establishes *who someone is*; it has no opinion on whether they may use this system. That decision is ours, made in one place — a custom `OidcUserService` that resolves the verified email against `app_user` and throws if there is no matching active row. A sign-in from any other Google account, institute or personal, is rejected without creating anything. Emails are normalised before comparison and matched only against the `email_verified` claim, never a self-asserted address.

2.2 What this removes, and what it adds

Choosing Google deletes PIN generation and hashing, lockout counters, a reset flow, credential rate limiting and a remember-me token table — roughly 30 lines of configuration and allowlist logic in place of about 180. None of those can fail if none of them exist, and there is no weak six-digit secret to defend.

The costs are real and worth stating rather than glossing:

New exposure	Assessment
Dependency on Google availability	Attenders cannot sign in if Google OIDC is unreachable. The manager always can, and can record work on their behalf via `act-as`
Authentication not integration-tested end to end	The authorization-code flow is Spring Security's code. What *we* wrote — the allowlist decision — is unit-tested directly; other tests use `@WithMockUser`
OAuth client secret, two redirect URIs	One environment variable and a one-time console setup (cloud URL and `localhost`). Scopes are non-sensitive, so no Google review is needed
Recovery is partly external; emails stored	If an attender loses their Google account the manager repoints the allowlist, but they must first have *some* working account. Their email is personal data (Sec. 7)

The manager's password is rate-limited to 10 attempts/minute/IP rather than locked out after N failures. Lockout on a single-administrator account is itself a denial of service: anyone who knows the username could lock out the only person able to undo it.

Revoking a lost phone is `active = false`, which blocks the next sign-in immediately and revokes that user's push subscriptions in the same action; the session expires within 12 hours.

2.3 Authorization matrix

Role *and* ownership are both checked. Role alone is never sufficient for a task action, otherwise any attender could resolve another's work and collect the points.

Capability	Manager	Attender
View all complaints; trigger ingestion; upload CSV	yes	no
Classify, assign, reassign, reject, reopen	yes	no
Start / inspect / record docket / resolve	via `act-as`	**own assignments only**
Claim special duties	no	yes
Log penalties, adjustments, post special duties	yes	no
View leaderboard	full	ranks + own detail
Provision users, edit the allowlist, deactivate accounts	yes	no
Export reports; edit lexicon, weights, SLA table	yes	no

`act-as` lets the manager perform an attender's transition during roll-call when that person cannot use their phone. It runs the identical state machine but audits `actor = manager, on_behalf_of = attender`; the points still go to the attender.

3 System Architecture

3.1 A modular monolith, deployed as one Spring Boot jar

One executable `.jar` (Spring Boot 3.x, embedded Tomcat), organised internally into bounded modules with explicit interfaces. This is a deliberate choice over microservices:

- At $\leq$50 complaints/day with 4 users, distribution would add network failure modes, deployment complexity and eventual consistency in exchange for scale we will never need.
- **Decisively:** our core correctness guarantees rest on *single-database transactions*. Splitting the ledger from the workflow would turn a uniqueness constraint into a distributed-transaction problem.
- Module boundaries are drawn so a component *could* be extracted later, and are enforced in CI by ArchUnit rather than left to discipline.

Within modules we use **ports and adapters** only where an external system is involved. This is not decoration: the ingestion port is what lets the Google Sheets adapter and the CSV fallback adapter share one normalization, dedupe and categorization pipeline instead of drifting into two code paths.

3.2 Context and processing pipeline

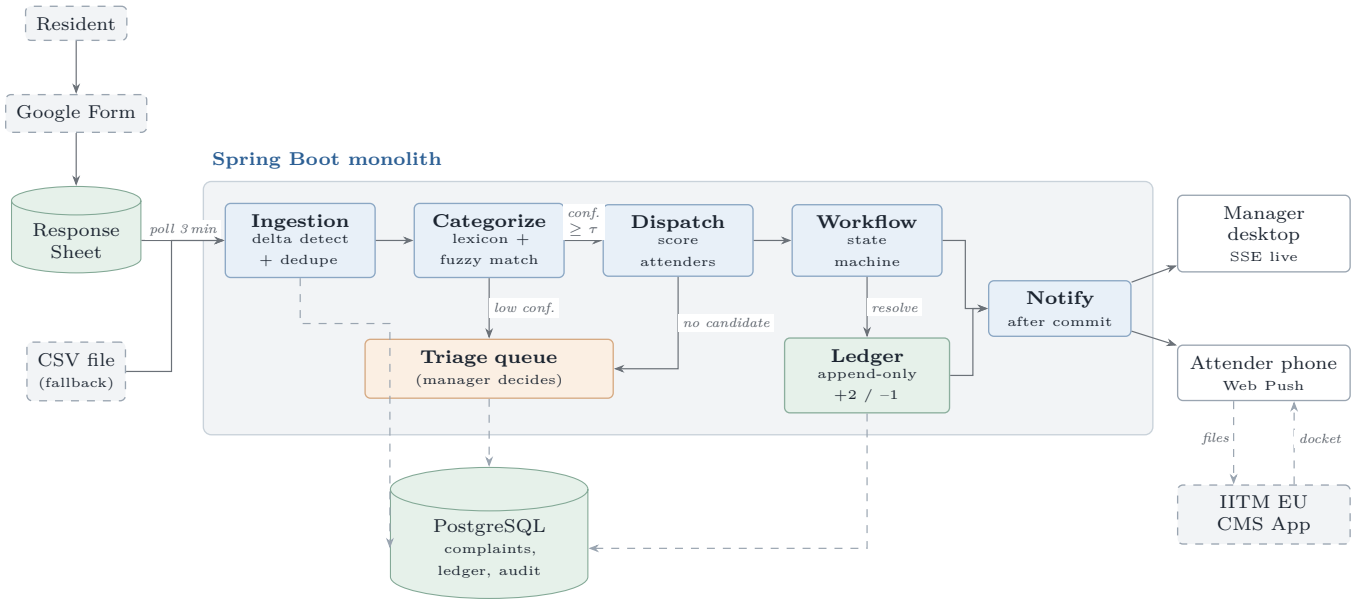


Figure 1: Context and processing pipeline. Dashed edges are external or persistence links; the IITM EU CMS app has *no* programmatic integration — the attender files there manually and types the returned docket number back into our system.

3.3 Module breakdown

Root package `in.ac.iitm.hostel.duty`. Each module owns exactly one invariant, which is what makes the boundary meaningful rather than cosmetic.

Module	Java package	Purpose & the invariant it owns
ingestion	`...ingestion`	Fetch rows, detect deltas, normalize into complaints. *A source row never becomes two complaints.*
categorization	`...categorization`	Assign domain + confidence to complaint text. *Low confidence never silently becomes a wrong domain.*
dispatch	`...dispatch`	Score attenders, produce assignments. *Every complaint gets an assignment or a recorded triage reason.*
workflow	`...workflow`	State machine, guards, transitions. *Only legal transitions occur, atomically with their side effects.*
ledger	`...ledger`	Point movements, leaderboard projection. *Append-only; no double-credit; balance is derived, never stored.*
notification	`...notification`	Outbox fan-out to manager SSE and attender Web Push. *Best-effort; never blocks a transaction.*
analytics	`...analytics`	MTTR, bottlenecks, anomaly detection. *Read-only over ledger and audit.*
reporting	`...reporting`	`.xlsx` generation via Apache POI. *Reports are reproducible point-in-time snapshots.*
identity	`...identity`	Users, roles, Google allowlist, manager password, provisioning. *No self-registration; every principal is pre-provisioned.*
audit	`...audit`	Append-only event log. *Written in the same transaction as the change it records.*

3.4 Layering

`web` → `application` → `domain`, with `infrastructure` implementing domain-facing ports. Four rules, enforced by ArchUnit in CI:

1. `domain` depends on nothing but the JDK — no Spring, no JPA behaviour.
2. `web` never touches a repository; it goes through `application`.
3. Cross-module calls use the target's published service interface, never its internals.
4. `@Transactional` lives only in `application`.

Rule 4 matters more than it looks: transaction boundaries are where the concurrency guarantees are enforced, so they must sit in one predictable place.

3.5 Deployment topology

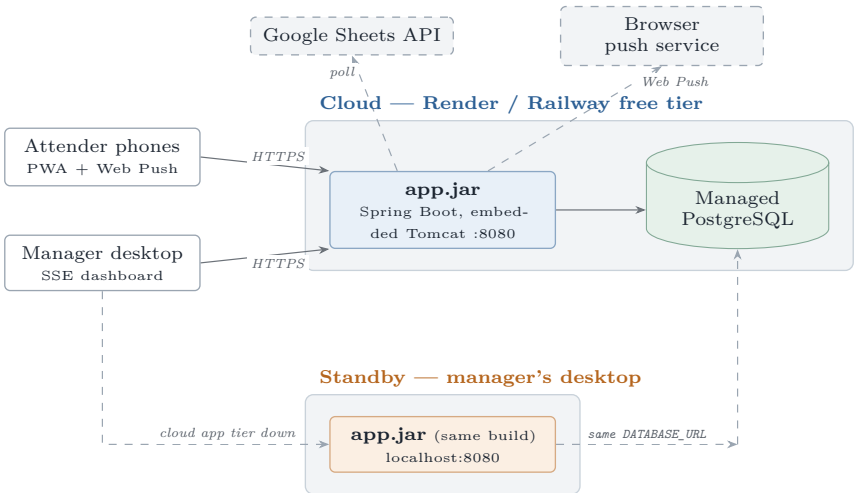


Figure 2: Deployment. **One database.** The standby is the same jar with the same `DATABASE_URL`, started on the manager's desktop when the cloud app tier is down. The outage designed for is the *platform* failing, not the network.

	Cloud (primary)	Standby (manager's desktop)
Database	Managed PostgreSQL	**The same** managed PostgreSQL
Reached by	Attenders and manager, HTTPS	Manager only, on `localhost`
Ingestion	Sheets polling + CSV	Both — internet is available
Notifications	SSE + Web Push	SSE + Web Push
Authentication	Google (attenders) + password (manager)	Manager password — no Google round-trip
Purpose	Day-to-day operation	Continuity when the app tier is down

Free-tier instances sleep when idle, so `@Scheduled` polling alone is unreliable. Mitigations: a **Sync Now** button that is the guaranteed path and also wakes the instance, and catch-up ingestion on startup — safe because delta detection is watermark-based rather than time-based, so a six-hour gap and a six-minute gap take the identical code path.

An earlier draft ran the standby on embedded H2 seeded from a nightly dump. That forced every migration into the SQL subset both engines accept, and left an unsolved problem: **work done during an outage had no path back into the cloud database.** Two diverging copies of the points ledger is precisely the failure this system exists to prevent — one database means it cannot arise. Residual risk is the managed database failing, which no application design fixes.

4 Data Model

4.1 Design principles

1. **Facts are immutable; state is derived.** Point balances are never a mutable counter — they are a `SUM` over an append-only ledger. This makes double-crediting a constraint violation rather than a race to win.
2. Every mutable aggregate root carries a `version` column for optimistic locking.
3. Audit rows are written *inside the same transaction* as the change they record, so the log can never disagree with the data.
4. No hard deletes on operational data; rows are tombstoned.

4.2 Entity–relationship model

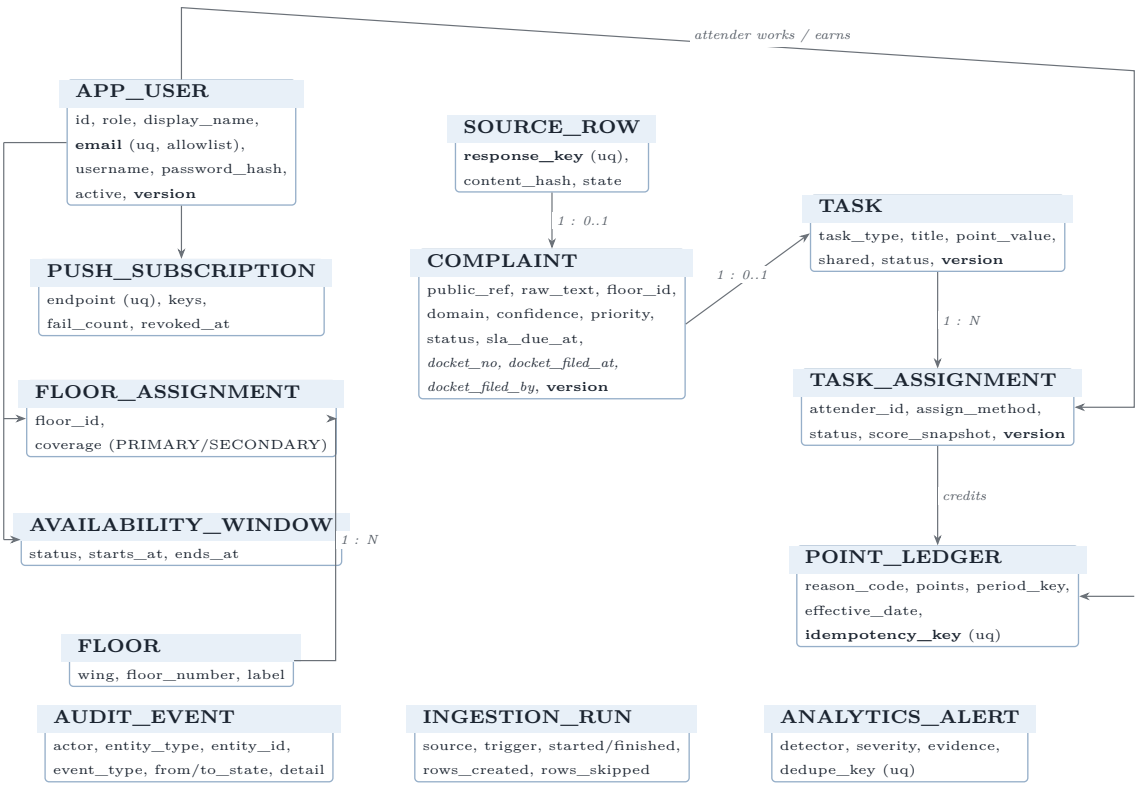


Figure 3: Core entities. Bold fields carry the guarantees: `version` for optimistic locking, `response_key` for ingestion identity, `idempotency_key` for point integrity.

4.3 The two constraints that carry the design

Constraint	What it prevents
`uq_source_row` (source, `response_key`)	A source row becoming two complaints. Identity is `SHA256(timestamp ‖ email)` — *not* row index, because deleting one sheet row shifts every index below it and a positional key would report the whole tail as amended.
`uq_ledger_idem` (`idempotency_key`)	Double-crediting. The key is derived from the *work* (`RESOLVE:{task}:{attender}`), not the request, so a retry on flaky hostel Wi-Fi — the expected case, not an exotic one — collides and is swallowed as a no-op.

Consequence. There is no `attender.points` column anywhere in the schema. No counter to increment means no lost update on a counter, and nothing to reconcile. A balance is a `SUM` over immutable facts, which is also what lets a bonus dispute be answered line by line.

4.4 Audit log

Every state change, point movement, authentication attempt and configuration change is written to `audit_event`. It is the evidence base for bonus disputes, so it is designed to be trustworthy rather than decorative.

Column	Content
`occurred_at`	UTC instant of the change
`actor_id`, `actor_role`	Who did it. `NULL` for system/scheduler actions
`on_behalf_of`	Set when the manager used `act-as`; the points still go to the attender
`entity_type`, `entity_id`	What was changed — `COMPLAINT`, `TASK`, `LEDGER`, `APP_USER`, ...
`event_type`	`TRANSITION`, `POINTS_CREDITED`, `PENALTY_LOGGED`, `SIGN_IN`, `SIGN_IN_REJECTED`, `USER_PROVISIONED`, `CONFIG_CHANGED`, ...
`from_state`, `to_state`	For transitions, the exact edge taken
`detail`	JSON payload: reason text, dispatch score snapshot, old/new config values
`correlation_id`	Groups everything caused by one request or one ingestion run

Two properties make it dependable. It is written by a `BEFORE_COMMIT` transactional event listener, so it **commits atomically** with the change it records and can never silently diverge. And there is **no code path that updates or deletes an audit row** — the JPA entity exposes no setters and the repository no delete method. Combined with the append-only ledger, an attender's monthly bonus can be reconstructed from first principles and defended line by line.

5 Complaint & Task Lifecycle

Status is the most-read field in the system: it drives the attender queue, the dashboard, SLA aging, MTTR measurement and the point credit. Leaving it as an informally-updated string is how a ticket ends up `RESOLVED` with no docket, or credited twice.

The *only* way to change it is `WorkflowService.transition(id, event, actor, payload)`, which loads the aggregate with its `version`, rejects any `(state, event)` pair absent from the transition table, evaluates the guard, then applies the change, its side effects and the audit row **in one transaction**; notifications fire separately on `AFTER_COMMIT`. Controllers, the scheduler and the CSV importer all funnel through it.

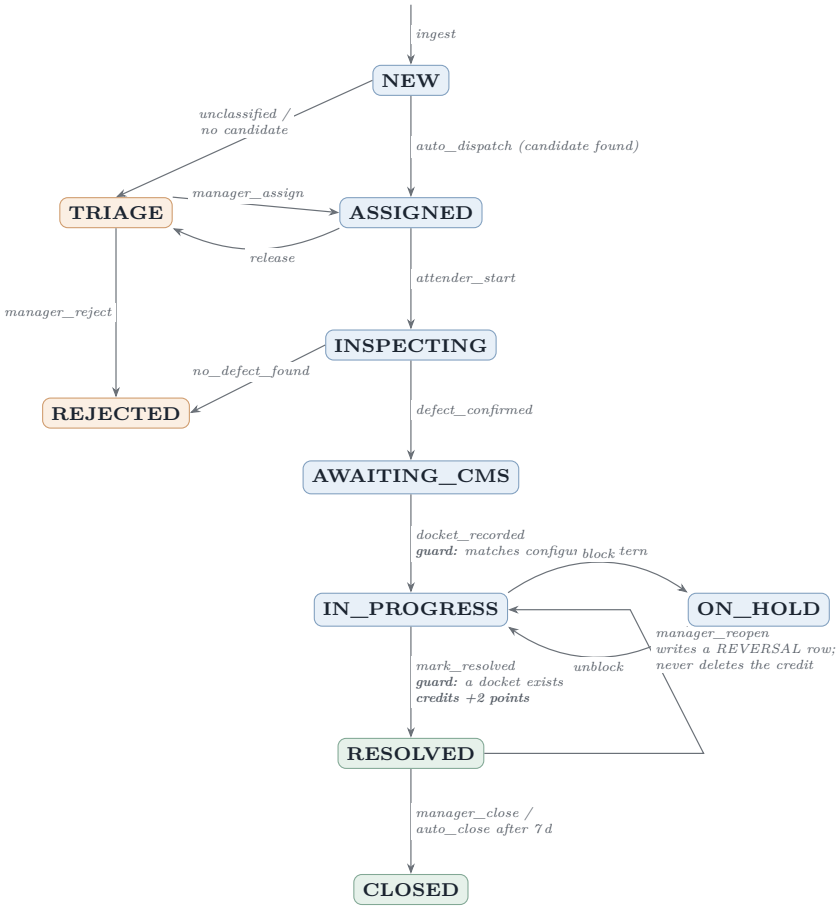


Figure 4: Complaint lifecycle. Two guards carry the institutional requirements: a docket must match the configured pattern to be recorded, and **a complaint cannot be resolved for points without one**.

5.1 Two guards worth calling out

The docket gate. `mark_resolved` requires a docket to exist. This makes institutional traceability structurally enforced rather than a matter of attender diligence — you cannot collect +2 points for a complaint you never filed with the Engineering Unit. Because there is no CMS API, we validate *format* only, and the pattern is configuration, so a wrong guess costs a property change rather than a redeploy.

Reopen reverses, never deletes. The ledger is append-only, so a reopen writes a compensating `REVERSAL` row; both the original credit and the reversal stay visible. Re-resolving afterwards writes a new credit under key `RESOLVE:{task}:{attender}:r{n}`, so the idempotency guarantee survives the reopen cycle without ever blocking legitimate repeat work.

5.2 Special duties

Special duties have no complaint and no docket: `OPEN` → `CLAIMED` → `COMPLETED`, with `CANCELLED` and `EXPIRED` exits. For a **shared** duty several attenders hold active assignments at once; the duty completes when every assignee has marked their part done, and *each* is credited once under their own key `DUTY:{task}:{attender}`.

5.3 Priority logic

Priority is not a fixed label — it ages, so old low-priority work eventually surfaces above new noise.

Base	Assigned when	Initial SLA
`P1`	Safety keywords matched **exactly** — *shock, spark, short circuit, sewage, no water, overflow.* No fuzzy matching here: a false P1 from a near-miss spelling would distort the whole queue.	Electrical 4 hOther 8 h
`P2`	Default for a confidently classified complaint	Elec 8 h, RO 12 h,Civil / Network 24 h
`P3`	Cosmetic keywords, or downgraded by the manager	72 h

A scheduled job escalates `P3` → `P2` → `P1` once a complaint has been open past **1.5**× its SLA window without

reaching `RESOLVED`. `base_priority` is retained, so the dashboard can always show "P1 (aged from P3)". Escalation caps at `P1`, and **hold time does not count toward it** — otherwise "waiting three days for the Engineering Unit to send a plumber" would be recorded as attender slowness and would point the bottleneck analysis at exactly the wrong place.

6 Dispatch Heuristic

Given a classified complaint c and attenders A, choose who should do the work. The formulation must always terminate with an assignment *or* a recorded triage reason, be deterministic including tie-breaks, be explainable to the manager, avoid starvation, and be tunable without a redeploy.

6.1 Stage 1 — hard eligibility filter

$$E = \{\, a \in A \;:\; a.\text{active} \;\wedge\; \text{availability}(a, \text{now}) = \texttt{AVAILABLE} \;\wedge\; \neg\,\text{assigned}(a, c) \,\}$$

Availability is a **gate, not a score**. Someone on leave must never be chosen no matter how well they score elsewhere; folding it into a weighted sum would let a strong floor match outvote the fact that a person is not at work. If $E = \varnothing$ the complaint goes to `TRIAGE` with reason `NO_AVAILABLE_ATTENDER` — the proposal's "all floor attenders unavailable" edge case, handled by construction rather than a special branch.

6.2 Stage 2 — score the eligible set

$$\text{Score}(a, c) = w_f \cdot \text{Floor}(a, c) \;+\; w_l \cdot \text{Load}(a)$$

Two terms, each normalised to $[0, 1]$, weights summing to 1, so Score $\in [0, 1]$ and each weight reads directly as "how much does this factor matter".

Term	Weight	Definition
Floor proximity	$w_f = 0.60$	`PRIMARY` coverage $= 1.00$; `SECONDARY` $= 0.70$; adjacent floor same wing $= 0.40$; same wing $\lvert\Delta\rvert \geq 2$ is $\max(0.10,\ 0.35 - 0.05\lvert\Delta\rvert)$; different wing $= 0.10$. Non-zero for distant attenders on purpose: when the primary is away someone still has to go.
Load (free capacity)	$w_l = 0.40$	$L(a) = \sum_t \pi(t)$ over active tasks with $\pi(\text{P1}){=}3, \pi(\text{P2}){=}2, \pi(\text{P3}){=}1$; $\text{Load}(a) = 1 - \min(1, L(a)/L_{cap})$, $L_{cap} = 8$. Weighting by priority reflects reality — three P1 electrical faults is not three P3 cosmetic complaints. `ON_HOLD` counts 0.

Two terms were cut, and the reason generalises. An earlier draft added a *domain skill* term and a *continuity* term. Skill defaulted to a neutral 0.5 for every attender, because we do not know whether attenders specialise — and a constant added identically to every candidate **cannot change the ranking**. It was arithmetic that looked like intelligence. Continuity (has this attender worked this room lately?) carried 10% weight and cost a 14-day lookback query per candidate; the signal it captures is used to better effect by the chronic-defect detector, which tells the manager to *replace the fixture* rather than merely routing the next complaint to the same person. What remains is a dozen lines of pure Java with no framework involvement.

6.3 Stage 3 — select, tie-break, record

best $= \arg\max_{a \in E} \text{Score}(a, c)$, with ties broken in strict order: (1) fewest tasks assigned *today* — the anti-starvation term; (2) longest time since last assignment; (3) lowest user id, which guarantees full determinism.

The complete score breakdown is persisted to `task_assignment.score_snapshot` as JSON (chosen attender, each component, the weights, runners-up, tie-break used). This is what makes "why did Ravi get this?" answerable months later, and it is what the dashboard renders in the assignment tooltip.

6.4 Degenerate cases

Case	Outcome
No attender available ($E = \varnothing$)	`TRIAGE` / `NO_AVAILABLE_ATTENDER`
Room number unparseable	`TRIAGE` / `UNMAPPED_LOCATION` — dispatching on a location we could not read is worse than asking
Floor exists, nobody covers it	Assign least-loaded, warn `UNCOVERED_FLOOR`
Every attender at capacity	Assign best anyway, notify `CapacityExceeded`. Overflowing visibly beats triaging silently
Complaint `UNCLASSIFIED`	Never reaches dispatch; triaged at classification
Manager disagrees	`manager_reassign`; both assignments audited

Why greedy, not optimal assignment? A global optimum (Hungarian, min-cost flow) is rejected deliberately. Complaints arrive *one at a time* from a 3-minute poll, so a typical batch is 1–3 items — and the optimal assignment of a one-item batch *is* the greedy one. With $|A| = 4$ the theoretical gain is negligible, while batch optimisation would mean holding complaints back to accumulate a batch, adding latency to exactly the P1 cases that matter most. Greedy is also arguable: "Ravi 0.78 vs Kumar 0.50" is something a manager can push back on.

7 PII & Data Protection

Complaints originate from residents, so the system holds personal data about people who never agreed to use it and cannot log in to check what we keep. That asymmetry drives a minimisation-first stance.

Data	Why held	Retention
Resident email	Contacting the reporter when details are unclear	90 days, then nulled
Room number	**Operationally essential** — dispatch depends on it	1 year
Complaint text	Diagnosis, classification, chronic-defect detection	1 year, then `[purged]`
Attender name	Assignment, leaderboard	Life of employment
Attender Google email	Allowlist key; identifies the actor in the audit log	Life of employment
Push endpoint + keys	Delivering assignment alerts	Until revoked or 3 failures
Audit log, point ledger	Bonus dispute resolution	Indefinite

What we deliberately do not collect. Resident names and phone numbers are simply omitted from the ingestion column map, even when the Google Form offers them. The cheapest way to protect personal data is not to hold it.

Aggregates survive purging. MTTR, arrival counts and chronic-defect detection all depend on timestamps, domain and room — not on what the resident wrote — so analytics keeps working after text redaction. Similarly, `source_row.raw_payload` is purged at 90 days while `content_hash` is retained, so we can still recognise a previously-seen row without storing what it said: retention and idempotency coexist rather than trade off.

Push payloads are end-to-end encrypted (`aes128gcm`), so Google's or Mozilla's relay sees only ciphertext. Notification bodies still name *a room, never a person* — a lock-screen preview is visible to anyone holding the phone.

Test fixtures are anonymised at extraction, before real data ever enters the repository. A committed fixture is effectively permanent, so the scrub cannot be a later step: emails become `resident{n}@example.com`, names are removed, and room numbers are remapped through a fixed permutation that preserves floor structure (keeping dispatch tests meaningful) while breaking identifiability. The mapping table is not committed.

Secrets live only in environment variables — the Google service-account key (base64), `DATABASE_URL`, `APP_SECRET`, and the VAPID key pair. `.gitignore` blocks the obvious filenames and CI runs Gitleaks on every pull request, so a committed credential fails the build rather than reaching `main`. The Sheets service account is granted **read-only** access to a single sheet.

8 Technology Stack by Module

Language Java 21 • **Build** Maven • **Framework** Spring Boot 3.x • **Database** PostgreSQL 16 • **Packaging** single executable `.jar`

Module	Libraries & technology	Milestone
identity	Spring Security 6: OAuth2 client (Google OIDC) for attenders, form login + BCrypt cost 12 for the manager, custom `OidcUserService` for the allowlist	S1 *(12–18 Sep)*
persistence	Spring Data JPA / Hibernate 6, Flyway (forward-only migrations), HikariCP, Testcontainers for tests	S1
ingestion	`google-api-services-sheets:v4` + service-account auth; Apache Commons CSV for the fallback adapter; SHA-256 via JDK `MessageDigest`	S2 *(19–25 Sep)*
categorization	Hand-built weighted lexicon; Apache Commons Text (Levenshtein, Jaro–Winkler) for fuzzy matching. *Apache OpenNLP `DocumentCategorizer` deferred to an optional second stage*	S2
dispatch	Plain Java domain code, no framework. Weights bound via `@ConfigurationProperties`	S3 *(26 Sep–2 Oct)*
workflow	Hand-rolled transition table over a Java `enum`. *Spring Statemachine was considered and rejected — our guards need transactional side effects, which is simpler to express directly*	S3
ledger	JPA with `@Version`; correctness carried by a database `UNIQUE` constraint rather than application logic	S3
scheduling	Spring `@Scheduled` / `@Async`; ShedLock for distributed lock safety	S2–S3
web (manager)	Thymeleaf, Tailwind CSS (purged), Chart.js 4	S4 *(3–9 Oct)*
web (attender)	Thymeleaf server-rendered shell + PWA service worker (push receiver)	S4
validation	Hibernate Validator (Jakarta Bean Validation) — entity constraints, CMS docket pattern	S3–S4
deployment	Docker Compose locally; `.jar` on Render / Railway with managed PostgreSQL	S4
concurrency	JPA optimistic locking; `SELECT ...FOR UPDATE` for duty claims only	S5 *(10–16 Oct)*
analytics	Plain Java statistics (median, MAD, EWMA, Poisson tail) over JPQL aggregates. *No stats library — the five estimators we need are a few lines each and stay explainable*	S6 *(17–23 Oct)*
reporting	Apache POI (`poi-ooxml`) for multi-tab `.xlsx` export	S7 *(24–30 Oct)*
notification	Spring MVC `SseEmitter` on `AFTER_COMMIT`; `nl.martijndwars:web-push` with VAPID for attender push	S7
testing	JUnit 5, AssertJ, Mockito, Testcontainers, ArchUnit, JaCoCo	Throughout
CI/CD	GitHub Actions (`mvn verify`), Gitleaks, OWASP Dependency-Check, Dependabot	S1

9 Test Plan

Level	Tooling	Scope	Share
Unit	JUnit 5, AssertJ	Pure domain logic — scoring, classification, transitions, statistics. No Spring context	70%
Slice	`@DataJpaTest`, `@WebMvcTest`	Repositories against real Postgres; controller contracts	15%
Integration	`@SpringBootTest` + Testcontainers	Full pipeline, transactions, concurrency	10%
Concurrency	`CountDownLatch`, `ExecutorService`	The five named races	named suite
Architecture	ArchUnit	Layering rules of Sec. 3.4	continuous
Acceptance	Stakeholder walk-through	The manager's own criteria	per milestone

All database tests run against real PostgreSQL via Testcontainers, never an in-memory substitute. The properties we most need to verify — unique-constraint behaviour under concurrent inserts, optimistic-lock failures, row locking — are precisely the ones an H2 stand-in would fake convincingly and wrongly. Coverage floor: 75% line, 90% on `domain`, gated in CI.

9.1 Fixtures

	Fixture	Purpose
F1	∼200 real complaints, anonymised	Classifier gold set; realistic dispatch
F2	∼60 hand-authored edge cases	Corner-case suite
F3	4 attenders × 25 tasks	Concurrency races
F4	90 days, ∼1800 complaints	Analytics and anomaly detectors
F5	∼30 malformed sheet rows	Ingestion robustness

9.2 Proving the 90% categorization criterion

The acceptance criterion needs a definition that cannot be gamed, so we fix one now. F1 is labelled by hand and split **70/30 into development and holdout**; the lexicon and thresholds are tuned *only* against development, and the holdout is scored once per milestone — tuning against it would produce a number that means nothing.

$$\text{coverage} = \frac{\text{auto-accepted}}{\text{total}} \qquad \text{accuracy} = \frac{\text{correct}}{\text{auto-accepted}}$$

Both are always reported together, because accuracy alone is trivially gameable: a classifier that refuses to classify anything scores 100%. Targets: **accuracy ≥ 90%, coverage ≥ 70%**, with a per-domain confusion matrix. Items a human labelled `AMBIGUOUS` are expected to reach triage; auto-accepting one counts as an error. In production every manager triage correction is a free `(text, true label)` pair, giving a growing corpus for monthly drift re-scoring.

9.3 Corner cases (selected from 31)

Case	Expected
Identical simultaneous submissions, two residents	Two complaints — distinct `response_key`
Byte-identical resubmission by one resident	One complaint — key collides
Same CSV uploaded twice by mistake	Zero duplicates
Sheet row deleted, shifting all indices below	**Zero spurious amendments**
Sheet row deleted entirely	Ignored — the sheet is an inbox, the complaint is unaffected
Required column renamed	Run fails loudly, never a silent partial ingest
Tamil-transliterated text (“thanni varala”)	Lexicon/fuzzy hit or triage — never a confident wrong domain
Resolve attempted with no docket	`GUARD_FAILED` 422
Resolve → reopen → resolve	Credit, reversal, new credit. Net +2, three ledger rows
Attender A acts on attender B’s task	403 `NOT_ASSIGNED`
Month boundary at 02:00 IST on the 1st	Falls in the new month
Anomaly detector below its n gate	Emits nothing
Push service returns `410 Gone`	Subscription revoked; no further attempts
Push unreachable for an hour	Retries, then `FAILED`. **No state transition affected**

9.4 Concurrency suite

Each test runs against real Postgres with threads released by a latch, and asserts on **final database state** rather than response codes.

	Setup	Assertion
X1	10 threads, one task, same idempotency key	Exactly 1 ledger row, +2 total, all 10 get 200
X2	10 threads, one task, *different* keys	Still exactly 1 ledger row
X3	Manager reassigns while attender resolves	One succeeds, loser gets 409, ledger matches the winner
X4	4 attenders complete a shared duty at once	4 ledger rows, task completed exactly once
X5	4 attenders claim a 1-slot duty at once	1 assignment, 3 receive `NO_SLOTS_LEFT`
X6	3 parallel ingestion runs over the same rows	No duplicate complaints
X9	**4 attenders × 25 tasks concurrently**	100 ledger rows, 200 points, zero lost updates, p95 < 500 ms

X2 is the sharpest test in the suite. It proves correctness does *not* depend on the client sending a good idempotency key. Even a buggy client generating a fresh UUID per retry cannot double-credit, because the ledger key is derived from the work rather than the request.

9.5 What we are not testing

Browser matrix beyond Chrome on Android and desktop — that is what these five people use. Load beyond the stated envelope; there will never be a thousand users. The Google Sheets API and the browser push services themselves, both mocked at the port boundary: we test our reconciliation and failure handling, not Google's delivery. The IITM EU CMS app, since no integration exists. Penetration testing, which is out of scope for a five-account internal tool.